

Strängar

- Det finns ingen speciell sträng-typ i C. C är en räkka tecken som avslutas med ett null-tecken

```
char a[] = "abc";  
a[0] = 'a'  
a[1] = 'b'  
a[2] = 'c'  
a[3] = '\\0'
```



- I filen `string.h` finns funktionsprototyper för många användbara sträng-funktioner
 - `strlen`, `strcpy`, `strcat`, `strstr`, `strchr`

Exempel

```
#include <stdio.h>

int main(void)
{
    int nstudents = 0; /* Initialisering */

    printf("How many students does ÅA have ?:");
    scanf ("%d", &nstudents); /* Läs input */
    printf("ÅA has %d students.\n", nstudents);

    return 0;
}
$How many students does ÅA have ?: 5000 (enter)
ÅA has 5000 students.
$
```

Typkonverteringar

```
#include <stdio.h>
int main(void)
{
    int i,j = 12;        /* endast j initialiseras*/
    float f1,f2 = 1.2;

    i = (int) f2;         /* explicit: i <- 1, 0.2 lost */
    f1 = i;               /* implicit: f1 <- 1.0 */

    f1 = f2 + (float) j; /* explicit: f1 <- 1.2 + 12.0 */
    f1 = f2 + j;         /* implicit: f1 <- 1.2 + 12.0 */
}
```

Typkonvertering

- Överraskningar / feltänkt??

```
int a, b=1;  
a = 2 * (b / 2); /* a har nu värdet 0 */
```

MEN:

```
a = 2 * (b / 2.0); /* a har nu värdet 1 */
```

```
int sum=7, count=3;  
float average;
```

```
average = sum/count; /* ger medeltalet 2.0 */
```

MEN:

```
average = (float)sum/count; /* ger medeltalet 2.3333 */
```

Operatorer

- Arithmetiska
 - `int i = i+1; i++; i--; i *= 2;`
 - `+, -, *, /, %,`
- Relationer and logik
 - `<, >, <=, >=, ==, !=`
 - `&&, ||, &, |, !`
- Observera:
 - `=` tilldelning
 - `==` jämförelse

Konstruktioner

- **if (expr) statement**
 - Om värdet på **expr** är olika 0 exekveras **statement**
- **if (expr) statement else statement2**
 - Om värdet på **expr** är lika med 0 exekveras **statement2**

statement är antingen enkel: **a=2;** , eller sammansatt: **{a=2; b=1;}**

if - exempel

```
/*  
if.c  
*/  
#include <stdio.h>  
int main(void)  
{  
    int a = 2;  
    int b = 3;  
    if(a > b) printf("a är större än b!\n");  
    else printf("a är inte större än b!\n");  
    return 0;  
}
```

while

- **while (expr) statement**
 - Så länge värdet på expr är olika 0 exekveras statement
- **do statement while (expr)**
 - Annan version av while

for

- `for (pre_expr; loop_expr; post_expr) statement`
- Loop-konstruktion
 - Först utförs `pre_expr`
 - Så länge `loop_expr` är olika 0, utförs först `statement` och sedan `post_expr`
- Mycket effektiv konstruktion

for - exempel

```
int a,i,n=5;  
a=0;  
for (i=0; i<n; i++) a += i*i;
```

Ekvivalent med

```
int a,i,n=5;  
for (a=0,i=0; i<n; a += i*i++) ;
```

Funktioner

- Program delas in i mindre block
 - Modularitet – enklare att
 - debugga, koda
 - Återanvändning av kod
- Funktionsdeklaration – prototyp
- Funktionsdefinition – prototyp + kod = implementation

Funktionsdefinitioner

```
type name(parameter_list)
    {deklarationer sätser}
```

- Om en funktion inte returnerar något ges `void` som returtyp
- Parameterlistan består av typ + variabelnamn
(`int a, float b`)
- Vid definitionen kallas `a` och `b` för de formella parametrarna
- Vid anropet `f(x, 4)` kallas `x` och `4` för de aktuella parametrarna, eller argument

Funktionsanrop

- Före funktioner anropas rekommenderas att funktionen är deklarerad (dvs kompilatorn känner till funktionens prototyp)
 - returtypen är korrekt
 - parametertyperna kan valideras
 - detta görs vanligen genom att inkludera en header-fil

```
#include <stdio.h>
```
- Funktionsanrop = funktionens namn + parametrarna i parentes: **pow(2, 4)**
- Funktionsanrop sker alltid call-by-value !!

Funktioner - exempel

```
#include <stdio.h>
int sum(int a, int b);
    /* function prototype at start of file */

void main(void) {
    int total = sum(4,5); /* call to the function */

    printf("The sum of 4 and 5 is %d", total);
}

int sum(int a, int b){    /* the function itself
    - arguments passed by value*/
    return (a+b);        /* return by value */
}
```

Variabler – synlighet

- I C är grunden för variablers synlighet per fil = kompileringsblock
 - En variabel som är deklarerad utanför en funktion är *global* inom nuvarande kompileringsblock
 - En variabel som deklarerats inom en funktion är lokal inom funktionen
 - En variabel som deklarerats inom en konstruktion (**for**, **while**, **if**) är lokal inom konstruktionen

Synlighet - exempel

```
#include <stdio.h>

int gCommonPar; /* Global variabel */

void main(void){
    int i=0; /* Lokal inom funktionen */
    for (i=0; i<5;i++) {
        int j; /* Lokal inom kompileringsblocket */
        j=i*5+gCommonPar;
        printf("Beräkningen gav: %i\n", j);
    }
    j=3; /* Ger kompileringsfel, variabeln j finns
        ej inom detta block */
}
```


Variabler - livslängd

- **static**, **auto** definierar variablers livslängd
 - **static** – variabeln placeras i heapen, får en egen plats i minnet som existerar under programmetts hela exekvering
 - **auto** – variabeln placeras på stacken – minnesutrymmen finns bara så länge exekveringen är inom konstruktionen
 - default är **auto** för lokala variabler – **static** är default för globala variabler

Livslängd exempel

```
#include <stdio.h>

int gimmenext() {
    static int next=0; /* Heap */
    return ++next;
}

int gimmenext2() {
    int next=0; /* Auto(stack) */
    return ++next;
}

int main() {
    printf("%i\n",gimmenext());
    printf("%i\n",gimmenext());
    printf("%i\n",gimmenext2());
    printf("%i\n",gimmenext2());
}
```

% ./gimmenext
1
2
1
1

Andra lagringsklasser

- **register** – direktiv för kompilatorn att placera variabeln i ett processorregister
- **volatile** – direktiv för kompilatorn att inte utföra optimeringar kring denna variabel
- **extern** – variabeln finns definierad som global i ett annat kompileringsblock, global variabel delad mellan kompileringsblock
- **const** – variabel definieras som konstant, dvs kompilatorn skall ej godkänna modifieringar av variabelvärde

Exempel - extern

```
/* fil_a.c */
```

```
int gCommonPar = 5;
```

```
/* fil_b.c */
```

```
#include <stdio.h>
```

```
extern int gCommonPar;
```

```
int main() {
```

```
    printf("Parameter=%i\n", gCommonPar);
```

```
}
```

```
% gcc fil_a.c fil_b.c -o prog
```

```
% ./prog
```

```
Parameter=5
```

Strukturer

- C implementerar strukturer

```
struct t_person {  
    char namn[30];  
    int alder;  
    int skonummer;  
};  
int main() {  
    struct t_person person; /* Strukturen på  
    stacken */  
    person.alder = 23; /* Adressering via "." */  
    strcpy(person.namn, "kalle");  
}
```

Nästade strukturer

```
#define MAX_DELTAGARE 100
struct t_kurs {
    char namn[30];
    int nDeltagare;
    struct t_person deltagare[MAX_DELTAGARE];
};

main() {
    struct t_kurs kurs;
    strcpy(kurs.namn, "Programmering i C/C++");
    kurs.nDeltagare = 1;
    kurs.deltagare[0].alder = 77;
    strcpy(kurs.deltagare[0].namn, "Axel Eklund");
}
```

Synonymer för datatyper

```
typedef unsigned short WORD;
```

```
WORD wIndex; /* samma som unsigned short wIndex */
```

```
typedef struct t_person Person;
```

```
Person jag; /* som struct t_person Jag; */
```

```
typedef struct t_point {  
    WORD x;  
    WORD y;  
} Point;
```

```
Point point;  
point.x = 12; point.y=44;
```